DocAssist AI

Conversational appointment booking with a clinic management dashboard

Speak naturally.
Let the backend validate every booking.

Next.js + React • FastAPI • PostgreSQL • Vapi

Architecture, features, voice providers and cost planning

The problem we address

These are the project’s design motivations; no measured business outcomes are claimed.

Patient experience

- Booking requires staff availability.
- Dates and clinic hours can be misunderstood.
- Patients need a simple way to ask, book or cancel.

Clinic operations

- Repetitive scheduling interrupts staff.
- Manual checks can miss holidays or conflicts.
- Staff need one place to inspect appointments.

Our solution and delivered features

The clinical guidelines are reference content; the system is not presented as a diagnostic assistant.

Patient-facing

- Browser-based voice receptionist
- Appointment booking and cancellation
- Availability-related backend tools
- Spoken confirmations and validation explanations

Staff-facing

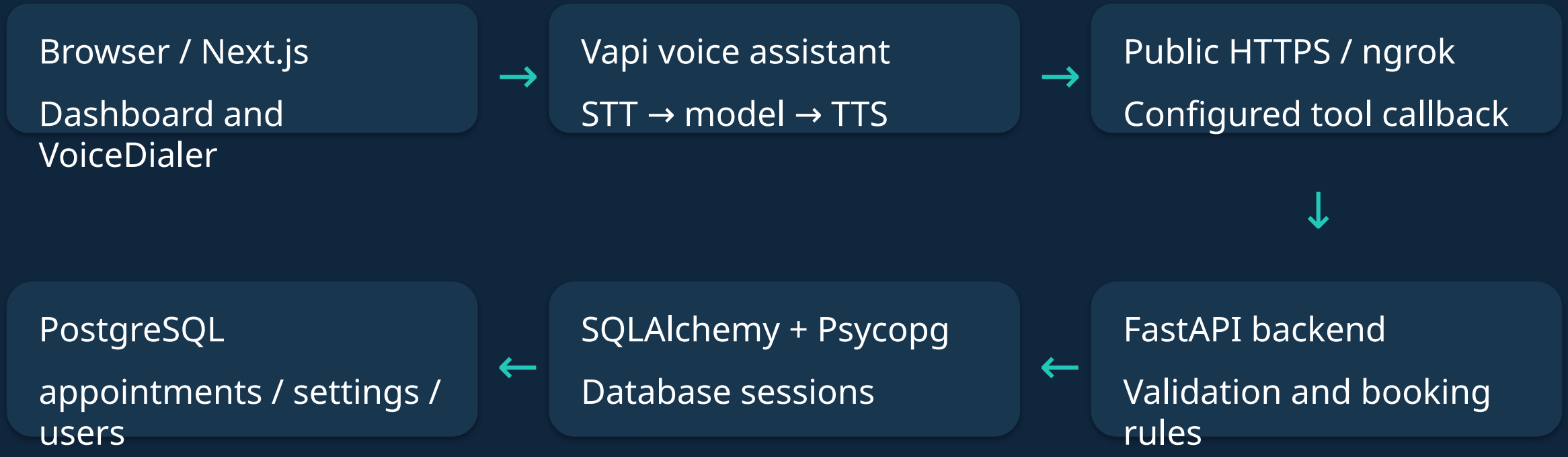
- Dashboard and appointment list
- Clinic hours, working days and holidays
- Configurable slot-duration setting
- Signup/login, account page and guidelines

Technology stack

Layer	Technology	Why it fits
Frontend	Next.js, React, TypeScript	Interactive pages and reusable typed components
UI / charts	Tailwind CSS, Recharts, Lucide	Consistent interface and appointment summaries
API	Python, FastAPI, Uvicorn, Pydantic	Request validation, tools and business logic
Persistence	SQLAlchemy + Psycopg + PostgreSQL	Relational models, sessions and durable records
Voice	Vapi Web SDK + configured providers	Audio sessions, transcripts and tool calls
Development	pgAdmin + optional ngrok	Database inspection and public tool callbacks

PostgreSQL is the user-confirmed local database. The repository also supports SQLite fallback.

Architecture: separate conversation from booking authority



The dashboard also calls FastAPI directly over HTTP/JSON. pgAdmin connects directly to PostgreSQL.

The Vapi tool URL and provider settings are managed outside the frontend repository.

Which GPT, STT and TTS are used?

Component	Selection in supplied project documentation	Confirmation status
Conversation model	OpenAI GPT-4.1	Reported in supplied documentation; verify active setting
STT / transcription	Soniox / Deepgram	Supplied list has alternatives; exact model unresolved
TTS / spoken voice	Elliot (Vapi Voice)	Supplied voice label; exact provider/model unresolved
Orchestration	Vapi + @vapi-ai/web	Confirmed in frontend source
Caller transport	Browser voice / WebRTC	Browser SDK integration; phone route not required

Do not describe these as fully verified active settings until the Vapi assistant configuration is checked.

How a voice booking works

01	**Listen**	Patient gives name, reason and appointment time.
02	**Understand**	STT produces text; the model selects a configured tool.
03	**Validate**	FastAPI checks the window, clinic schedule and conflicts.
04	**Save**	SQLAlchemy commits the accepted appointment to PostgreSQL.
05	**Respond**	Tool result returns to Vapi; TTS speaks the outcome.

Booking rules and data design

Implementation note: the current branch uses a calendar-date upper cutoff; the previously supplied update implements an exact rolling 240-hour window. Slot-duration settings do not alone prove overlap protection.

Requested booking policy

- Accept now ≤ requested time ≤ now + 10 days.
- Apply clinic timezone consistently.
- Reject holidays and closed working days.
- Enforce clinic hours and check existing bookings.

Stored records

- appointments: patient, reason, time and cancellation
- settings: hours, duration, days and holidays
- users: name, email and password hash

Why choose PostgreSQL?

PostgreSQL improves the persistence foundation; it does not automatically enforce every business rule.

Advantages for this project

- Shared relational storage for staff and API
- Transactional updates and concurrent access
- Constraints, indexing and backup options
- Can run independently of the backend

Tradeoffs and safeguards

- Requires a service, credentials and backups
- Migration must transfer existing SQLite records
- Check-then-insert can still race
- Add database-level booking protection and migrations

Why choose Vapi?

Vapi is a practical POC choice, not a claim of universal superiority over other platforms.

Reasons for selecting it

- Existing React Web SDK integration
- Managed voice-session orchestration
- Tools connect to FastAPI endpoints
- Call events and logs support debugging

Tradeoffs

- External-service and provider dependence
- Usage cost across several components
- Tool configuration and audio lifecycle need care
- Quality and latency require measured testing

Alternatives and their tradeoffs

Option	Advantages	Disadvantages
Vapi	Managed orchestration; existing SDK and tools	Provider dependency; component costs; lifecycle handling
Retell AI	Managed conversational voice-agent platform	SDK/tool migration; verify plan features and pricing
Bland AI	Managed phone-call automation workflows	Verify browser fit, coverage and workflow constraints
OpenAI Realtime API	Direct audio interaction and more session control	More auth, tool execution, reconnection and phone integration work
Twilio + AI services	Flexible telephony routing and provider choices	More integration; separate telephony, speech and model costs

Evaluate actual booking success, latency, cost and privacy with the same test scenarios.

Illustrative voice cost breakdown — USD

Component	README estimate / minute	How to interpret it
Vapi orchestration	$0.050	Supplied estimate; verify current plan
STT	$0.004	Exact provider/model must be confirmed
LLM / GPT-4.1	$0.020	Per-minute planning allowance; token usage varies
TTS / Elliot label	$0.020	Planning allowance; actual voice billing varies
Browser carrier leg	$0.000 assumed	No PSTN leg in this browser-only scenario
Total	$0.094 / minute	Arithmetic sum; not a verified live vendor quote

Excludes hosting, database service, ngrok plan, taxes and optional phone numbers/carrier calls. No current free-credit offer is guaranteed.

POC budget scenarios

Usage scenario	Calculation	Illustrative voice cost
One 2-minute call	2 × $0.094	$0.188
100 calls × 2 minutes	200 minutes × $0.094	$18.80
500 calls × 2 minutes	1,000 minutes × $0.094	$94.00
PKR conversion	USD total × actual USD/PKR rate	Use your billing-date exchange rate

Actual model cost depends on tokens and context; STT/TTS billing units and any bundled pricing must be checked to avoid double counting.

Demo plan and next steps

Demonstrate the workflow

1. Show clinic settings and available dates.
2. Request a valid voice booking.
3. Verify the row in pgAdmin.
4. Reject a past or too-distant booking.
5. Cancel a booking and show its status.

Before production

- Confirm GPT, STT and TTS settings in Vapi.
- Measure call cost, latency and success rate.
- Add server-side authorization and tool protection.
- Enforce concurrency-safe reservations.
- Set backups, stable HTTPS and monitoring.

Sources and configuration checks

Project evidence

- User-supplied project and pricing documentation
- test2-branch: README, frontend SDK/config, backend routes and SQLAlchemy models
- User-confirmed PostgreSQL migration and supplied rolling-window update

Official pricing / configuration references

- vapi.ai/pricing | docs.vapi.ai
- retellai.com/pricing | docs.retellai.com
- bland.ai | docs.bland.ai
- openai.com/api/pricing | platform.openai.com/docs/guides/realtime
- twilio.com/en-us/voice/pricing

Live vendor prices were not independently verified. Confirm provider IDs and costs in the Vapi assistant and call-level cost breakdown before presenting exact prices.